TinyRV1 Processor

Ayesha Rahman (ar2545)
Tiffany Li (tl885)

1 Introduction

Lab 4 gives us experience building a full TinyRV1 computer system by designing, testing, and prototyping both a single-cycle processor and an accumulate accelerator.

In Parts A through C, we build the processor step by step by implementing the immediate generator, ALU, register file, PC logic, and control signals. We then wire them together into a working datapath and control unit while checking each instruction against the control signal table from lecture. We also design a counter-based accelerator with an FSM controller, compare it to the software accumulate loop, and write assembly programs for the calculator, accumulate, and door monitor systems.

In Parts D and E, we bring the entire system onto the DE0-CV FPGA by wiring the memory bus, SPI loader, clock dividers, seven-segment displays, distance sensor, multi-note player, and the oscilloscope pin. We then run our programs on hardware to study real timing and behavior.

The purpose of this lab is to learn how a processor, accelerator, and embedded IO system work together and how software interacts with hardware through memory-mapped IO. This lab reinforces Topics 9 and 10 on ISA design and single-cycle processors and builds on hierarchy and modular design from earlier lectures.

2 Single-Cycle TinyRV1 Processor

2.1 Accumulate Assembly Program

```
addi x6, x1, 0
addi x4, x0, 0
addi x5, x0, 0x100

# accumulate loop
accum:
    lw   x7, 0(x5)
    add  x4, x4, x7
    addi x5, x5, 4
    addi x6, x6, -1
    bne  x0, x6, accum
```

Figure 1: Accumulate Loop

Figure 1 displays the accumulated assembly code and loop that was developed. It begins with three `addi` instructions initializing x6, x4, and x5.

x6 represents how many items need to be processed within the loop, so it is initialized to the value of x1, which represents the size of the array. x4 keeps track of the running total and begins at 0. x5 is used as a data pointer and stores the memory address of the current element.

The loop begins by loading the value at address x5 into x7 using `lw`. That value is added to the running total (x4 + x7), and the result is stored back into x4. Then x5 is incremented by 4 since each element is 4 bytes. Lastly, x6 is decremented by 1. The loop continues until x6 = 0, indicating that all elements have been processed.

2.2 FPGA Area Analysis

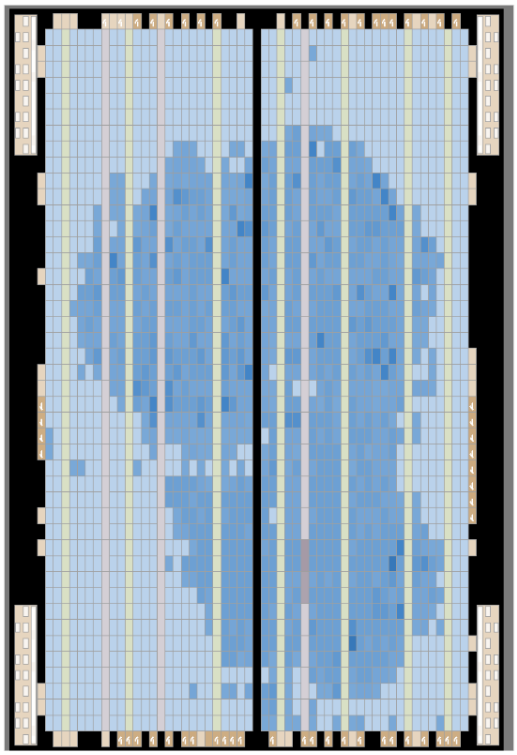

Figure 2: Chip planner showing location of logic used to implement processor

Figure 2 shows a large dark blue region in the center of the FPGA where the TinyRV1 processor is placed. Quartus reported:

- 6,244 ALMs
- 9,072 ALUTs
- 5,755 logic registers

This corresponds to about 34% of the DE0-CV's total fabric.
This large area usage makes sense because a single-cycle CPU includes:

- Register file
- Full ALU

- Immediate generator
- Next-PC logic
- Branch and jump hardware
- Memory bus
- SPI interface
- Full control logic

Compared to earlier labs:

- Lab 2 calculator used only a few hundred ALUTs
- Lab 3 music player used about 300–500 ALUTs
- TinyRV1 processor uses over 9,000 ALUTs

Quantitatively, the difference in area is a relatively big jump from our Lab 2 two-function calculator only used a few hundred ALUTs, and the Lab 3 music player sat in a tiny corner of the chip with maybe 300–500 ALUTs, while the TinyRV1 processor uses over 9,000 ALUTs, which is around 20–30 times more than both of those earlier designs. This huge increase is important because it shows how quickly the area grows when moving from small, task-specific hardware to a full general-purpose processor, and it explains why the chip planner now shows a larger, dark region for the CPU instead of the small clusters of blue we saw in Labs 2 and 3. Even though the processor can perform the same tasks as the Lab 2 calculator or the Lab 3 music player through specific instructions, previous labs were more area-efficient because they implement only the exact hardware needed for one job, while the TinyRV1 processor must support a full instruction set, a register file, memory accesses, branching, and a shared datapath that can execute many different instructions, which makes it much larger and less specialized.

2.3 FPGA Delay Analysis

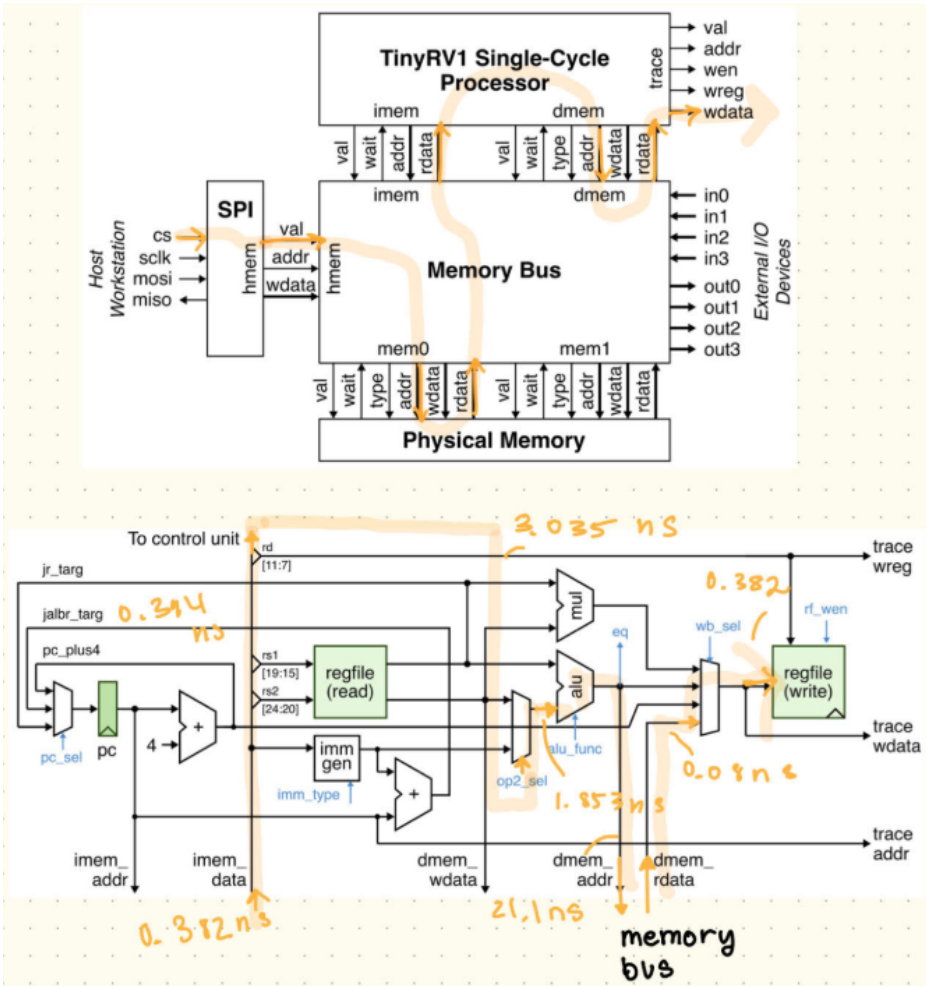

Figure 3: Embedded system diagram (top) and Processor Datapath Diagram (bottom)
with highlighted critical path and annotated delays

The system and processor datapath diagram are shown in Figure 3.

The system critical path starts at the SPI CS synchronizer, passes through the memory bus and physical memory, then returns to the processor through instruction memory and data memory.

Inside the processor datapath, the critical path corresponds to the `lw` instruction. It goes through:

- Control unit
- ALU
- Memory bus
- Data memory
- Write-back mux
- Register file

The processor required 159 cycles to accumulate 31 elements:

$$5 \text{ instructions} \times 31 + 4 \text{ setup instructions} = 159$$

The cycle time is 80 ns. The processor met timing at 63.280 ns according to the critical path report.

Using the formula:

$$\begin{aligned}\text{Time per program} = &(\text{instructions per program}) \times (\text{cycles per instruction}) \\ &\times (\text{time per cycle})\end{aligned}$$

$$= 159 \times 1 \times 80 = 12720 \text{ ns}$$

$$12720 \text{ ns} = 12.7\ \mu s$$

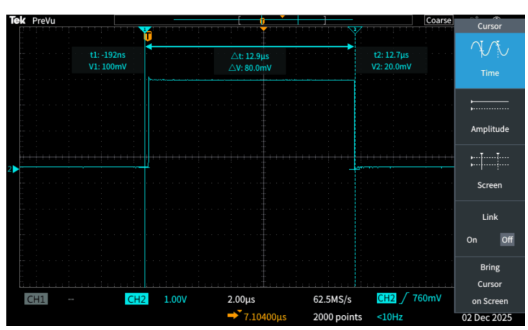

Figure 4: Oscilloscope waveform screenshot showing execution time in ns

The oscilloscope measurement in Figure 4 shows a period of 12.7 μs, which matches the calculated value.

3 Accumulate Accelerator

The accumulate accelerator sums an array of 32-bit values stored in memory.

Figure 5: Accumulate accelerator datapath diagram

The datapath (Figure 5) includes:

- Counter (for indices)
- Multiplier (index to byte address)
- Memory interface
- Adder (running sum)
- Register (stores sum)
- Control muxes

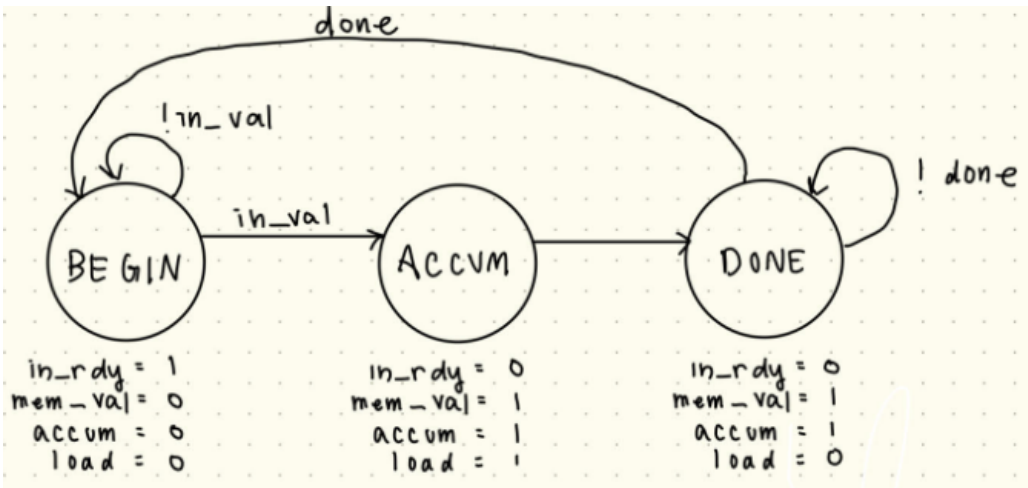

Figure 6: Accumulate accelerator FSM diagram

The FSM shown in Figure 6 has three states:

- BEGIN: Latches size and resets accumulator.
- ACCUM: Performs memory read and addition each cycle.
- DONE: Holds final result stable and waits for next request.

The interactive simulator produced 159 stat cycles for size 31. This matches the earlier 159-cycle calculation.

Qualitatively:

- The accelerator has a shorter critical path (counter, multiplier, adder).
- The processor has a longer critical path (decode, register file, ALU, mux trees).
- The accelerator uses much less FPGA area.

The processor must support the full TinyRV1 ISA, while the accelerator is specialized for one function.

4 Conclusion

Throughout this project, we learned how a full TinyRV1 system operates by designing both a single-cycle general-purpose processor and a specialized accumulate accelerator, and directly comparing their behavior, area, and timing on the FPGA. The accelerator demonstrated that specialized hardware will provide better efficiency for a task like accumulating, as it uses far less area and has a shorter critical path. In comparison, the general purpose processor consumed much more FPGA usage, and had a longer delay, creating a pareto optimal frontier. At the same time, the processor offered far greater flexibility and programmability, whereas the accelerator, although faster and smaller, was limited to one fixed function and required much more task-specific design work.

Overall, this project showed us how hardware and software interact at a low level, and it displayed the tradeoffs between generality and efficiency, tying together all of our learning taught in class. Our perspective of computer engineering now keeps in mind these tradeoffs, along with a strong understanding of how architectural decisions shape performance.

5 Appendix

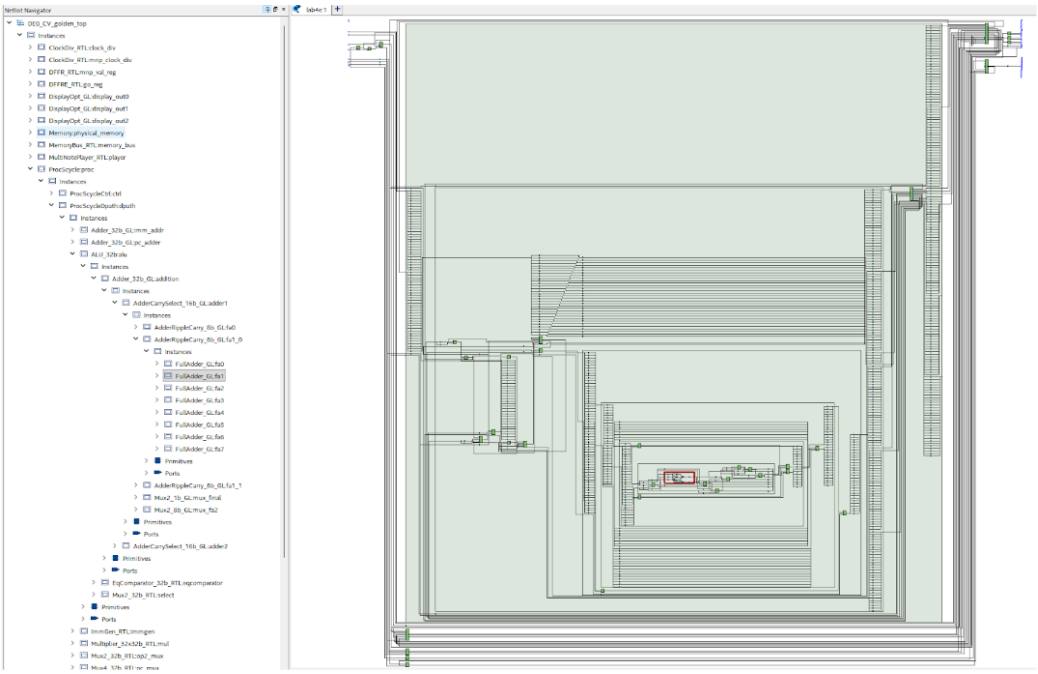

Figure 7: RTL viewer showing complete hierarchy on left and full adder gate-level implementation on the right for processor

Figure 8: Oscillopscope waveform screenshot that shows the SPI packets: the SCLK (blue) and MOSI (yellow) signals.